

Volumi e Tipi di Volume

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il concetto di volume in Kubernetes e come differisce dalla persistenza a livello di container
 - Descrivere **emptyDir** e i suoi casi d'uso: cache temporanea, condivisione dati tra container nello stesso Pod
 - Illustrare **hostPath** con i relativi rischi di sicurezza e gli usi legittimi in produzione
 - Configurare **configMap** e **secret** come volumi montati come file nel filesystem del container
 - Usare **projected** per combinare più sorgenti di configurazione in un singolo punto di mount
 - Scegliere il tipo di volume corretto in base al caso d'uso
-

Teoria e Concetti Chiave

Perché i Volumi?

In Kubernetes, il filesystem di un container è **effimero**: quando il container viene riavviato, tutti i file scritti durante la sua esecuzione vanno persi. I **volumi** risolvono questo problema fornendo un'area di storage con un ciclo di vita indipendente dal singolo container.

💡 **Suggerimento:** Il ciclo di vita di un volume è legato al **Pod**, non al container. Se un container nel Pod viene riavviato (crash, liveness probe), il volume sopravvive. Se il Pod viene cancellato e ricreato, i volumi di tipo **emptyDir** vengono persi, mentre i **PersistentVolume** sopravvivono.

Tipi di Volume Non-Persistenti

emptyDir

Un **emptyDir** è una directory vuota creata quando il Pod viene schedulato su un nodo e cancellata quando il Pod termina.

Caratteristiche:

- Inizialmente vuoto, scrivibile da tutti i container del Pod
- Sopravvive ai restart del container (ma non al delete del Pod)
- Di default usa il disco del nodo; con **medium: Memory** usa la RAM (tmpfs) — più veloce ma usa memoria del nodo

Casi d'uso:

- Cache temporanea per elaborazione dati
- Spazio di scambio tra container nello stesso Pod (es. sidecar che processa file prodotti dal container principale)
- Directory di lavoro per build temporanee

```
volumes:
- name: cache-vol
  emptyDir: {}          # disk-backed

- name: shared-mem
  emptyDir:
    medium: Memory      # RAM-backed (tmpfs)
    sizeLimit: 256Mi    # limite di memoria utilizzabile
```

hostPath

Un **hostPath** monta una directory o file dal **filesystem del nodo host** nel container.

Rischi di sicurezza:

- Il container può leggere/scrivere file sensibili del nodo host (es. `/etc/kubernetes`, `/var/lib/kubelet`)
- Richiede Security Context Constraint (SCC) **privileged** o **hostmount-anyuid** in OpenShift
- Rompe la portabilità del Pod (il path deve esistere su ogni nodo dove il Pod può essere schedulato)

⚠ Attenzione: Non usare **hostPath** per applicazioni utente in produzione su ARO. È accettabile solo per DaemonSet di sistema (es. agenti di logging che devono leggere `/var/log/containers/`).

Usi legittimi:

- DaemonSet per raccolta log (Fluentd, Filebeat): montano `/var/log/containers/`
- Agent di monitoraggio nodo: montano `/sys` o `/proc`
- Plugin CNI: montano `/etc/cni/net.d`

```
volumes:
- name: log-dir
  hostPath:
    path: /var/log/containers # path sul nodo host
    type: Directory          # Directory | File | DirectoryOrCreate | Socket
```

Tipi di Volume da ConfigMap e Secret

configMap come Volume

Monta le chiavi di un ConfigMap come file nella directory specificata. Ogni chiave diventa un filename; il valore diventa il contenuto del file.

Caso d'uso: Configurare applicazioni che leggono file di configurazione (es. `nginx.conf`, `application.properties`, `prometheus.yml`).

```
volumes:
- name: app-config
  configMap:
    name: webapp-config          # nome del ConfigMap
    defaultMode: 0644           # permessi dei file (ottale)
    items:                      # opzionale: monta solo alcune chiavi
      - key: app.properties
        path: config/app.properties # path relativo nel volume
```

secret come Volume

Monta le chiavi di un Secret come file. I dati del Secret (base64-encoded nell'etcd) vengono decodificati automaticamente prima di essere scritti nel filesystem del container.

Caso d'uso: Certificati TLS, file di configurazione con credenziali, chiavi SSH.

```
volumes:
- name: tls-certs
  secret:
    secretName: webapp-tls      # nome del Secret
    defaultMode: 0400          # permessi restrittivi (solo lettura owner)
    items:
      - key: tls.crt
        path: server.crt
      - key: tls.key
        path: server.key
```

 **Suggerimento:** I file montati da Secret/ConfigMap sono aggiornati automaticamente dal kubelet (con un leggero ritardo) quando il Secret/ConfigMap viene modificato, senza bisogno di riavviare il Pod.

projected Volume

Combina più sorgenti di volume (configMap, secret, serviceAccountToken, downwardAPI) in un **singolo directory di mount**.

Caso d'uso: Applicazioni che si aspettano tutti i file di configurazione in una singola directory.

```
volumes:
- name: all-config
  projected:
    defaultMode: 0444
    sources:
      - configMap:
          name: webapp-config
      - secret:
```

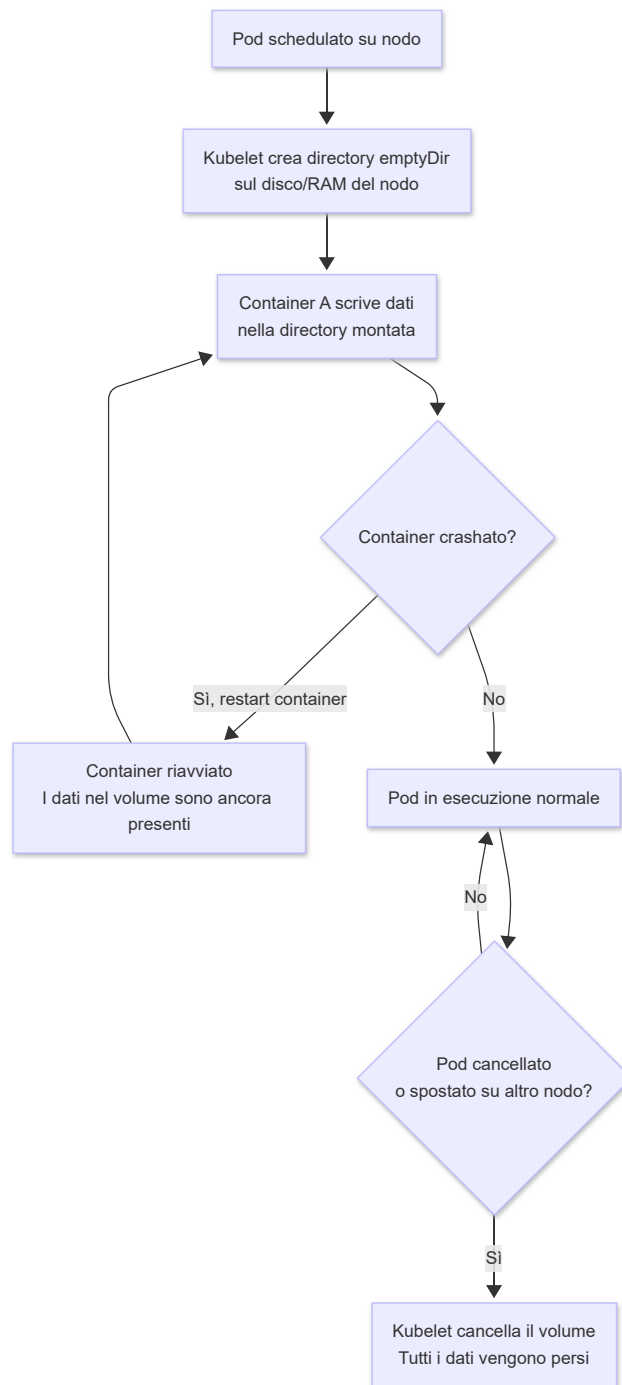
```
name: webapp-tls
- serviceAccountToken:
  path: token
  expirationSeconds: 3600
  audience: api
```

Tabella Comparativa dei Tipi di Volume

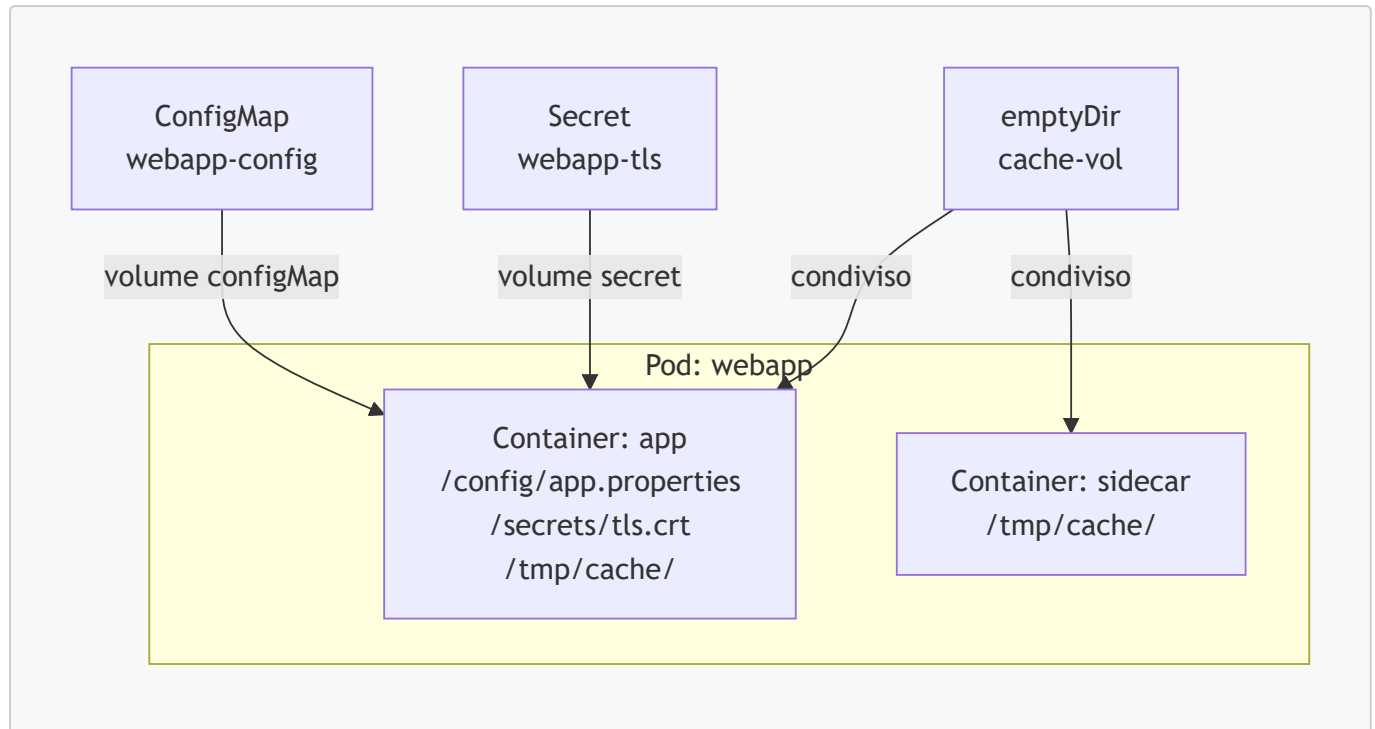
Tipo	Persistenza	Ambito	Casi d'uso principali	Limitazioni
emptyDir	Solo durante vita del Pod	Pod	Cache temporanea, condivisione tra container	Dati persi al delete Pod
emptyDir (Memory)	Solo durante vita del Pod	Pod	Cache in-memory ad alta velocità	Usa RAM del nodo; persi al delete Pod
hostPath	Permanente (sul nodo)	Nodo	DaemonSet di sistema, logging nodo	Non portabile; richiede SCC privileged
configMap	Duratura (finché ConfigMap esiste)	Cluster	File di configurazione applicazione	Read-only; max 1 MiB
secret	Duratura (finché Secret esiste)	Namespace	TLS certs, credenziali, chiavi SSH	Read-only; max 1 MiB; base64 in etcd
projected	Dipende dalle sorgenti	Namespace	Aggregare configMap + secret + token	Complessità configurazione
persistentVolumeClaim	Permanente	Cluster	Database, file applicativi persistenti	Richiede PV/StorageClass disponibile

Diagrammi

Ciclo di Vita di un emptyDir



Montaggio di Volumi in un Pod



Comandi Pratici con Output Atteso

```
# Visualizzare i volumi di un Pod in esecuzione
oc get pod webapp-7d8f9b-xkz2m -o jsonpath='{.spec.volumes[*].name}'

# Output atteso:
# cache-vol app-config tls-certs kube-api-access-7xkzm
```

```
# Descrivere i mount di un container nel Pod
oc describe pod webapp-7d8f9b-xkz2m | grep -A 5 "Mounts:"

# Output atteso:
# Mounts:
# /config from app-config (ro)
# /secrets from tls-certs (ro)
# /tmp/cache from cache-vol (rw)
# /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-7xkzm (ro)
```

```
# Verificare il contenuto di un volume configMap montato
oc exec -it webapp-7d8f9b-xkz2m -- ls -la /config/

# Output atteso:
# total 0
# drwxrwxrwx. 3 root root 95 May 24 10:00 .
```

```
# drwxr-xr-x. 1 root root 43 May 24 10:00 ..
# drwxr-xr-x. 2 root root 30 May 24 10:00 ..2026_05_24_10_00_00.12345678
# lrwxrwxrwx. 1 root root 31 May 24 10:00 ..data ->
..2026_05_24_10_00_00.12345678
# lrwxrwxrwx. 1 root root 18 May 24 10:00 app.properties -> ../data/app.properties
```

```
# Leggere un file montato da ConfigMap
oc exec -it webapp-7d8f9b-xkz2m -- cat /config/app.properties

# Output atteso:
# database.host=mysql.database-ns.svc.cluster.local
# database.port=3306
# app.log.level=INFO
```

```
# Verificare i permessi di un Secret montato come volume
oc exec -it webapp-7d8f9b-xkz2m -- ls -la /secrets/

# Output atteso:
# total 0
# -r-----. 1 root root 1234 May 24 10:00 server.crt
# -r-----. 1 root root 678 May 24 10:00 server.key
```

```
# Verificare l'utilizzo di un emptyDir (RAM)
oc exec -it webapp-7d8f9b-xkz2m -- df -h /tmp/cache/

# Output atteso (emptyDir con medium: Memory):
# Filesystem      Size  Used Avail Use% Mounted on
# tmpfs           256M    0 256M   0% /tmp/cache
```

```
# Creare un ConfigMap da un file di configurazione
oc create configmap webapp-config --from-file=app.properties=./app.properties -n
myproject

# Output atteso:
# configmap/webapp-config created
```

```
# Verificare le chiavi di un ConfigMap (che diventano filename nel volume)
oc get configmap webapp-config -o jsonpath='{.data}' | python3 -m json.tool

# Output atteso:
# {
```

```
# "app.properties": "database.host=mysql...\napp.log.level=INFO\n"
# }
```

Esempi YAML Completi

Pod con emptyDir, ConfigMap e Secret come Volumi

```
apiVersion: v1
kind: Pod
metadata:
  name: multi-volume-pod
  namespace: myproject
  labels:
    app: webapp
spec:
  containers:
    - name: app
      image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
      command: ["/bin/sh", "-c", "sleep 3600"]
      resources:
        requests:
          memory: "128Mi"
          cpu: "100m"
        limits:
          memory: "256Mi"
          cpu: "500m"
      volumeMounts:
        # Mount del ConfigMap come directory di configurazione
        - name: app-config
          mountPath: /config # ← ogni chiave del ConfigMap diventa un file
                               in /config/
          readOnly: true
        # Mount del Secret come directory dei certificati TLS
        - name: tls-certs
          mountPath: /secrets/tls # ← ogni chiave del Secret diventa un file in
                                   /secrets/tls/
          readOnly: true
        # Mount emptyDir come cache temporanea condivisa con il sidecar
        - name: cache-vol
          mountPath: /tmp/cache # ← directory scrivibile condivisa tra
                                   container del Pod
        - name: sidecar
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          command: ["/bin/sh", "-c", "sleep 3600"]
          resources:
            requests:
              memory: "64Mi"
              cpu: "50m"
            limits:
              memory: "128Mi"
```



```
    cpu: "200m"
  volumeMounts:
    # Il sidecar legge dalla cache prodotta dal container principale
    - name: cache-vol
      mountPath: /tmp/cache          # ← stesso volume, stesso path
  volumes:
    # emptyDir disk-backed: dati temporanei, condivisi tra container del Pod
    - name: cache-vol
      emptyDir:
        sizeLimit: 1Gi              # ← limita la quantità di disco usabile
    # ConfigMap come volume: monta tutte le chiavi come file
    - name: app-config
      configMap:
        name: webapp-config         # ← nome del ConfigMap nel namespace
        defaultMode: 0644          # ← permessi ottali per i file (rw-r--r--)
    # Secret come volume: monta le chiavi selezionate con permessi restrittivi
    - name: tls-certs
      secret:
        secretName: webapp-tls      # ← nome del Secret nel namespace
        defaultMode: 0400           # ← solo lettura owner (r-----)
        items:
          - key: tls.crt             # ← chiave nel Secret
            path: server.crt         # ← nome file nel volume
          - key: tls.key
            path: server.key
```

Note Specifiche per Azure ARO

hostPath e SCC su ARO

Su ARO, i Pod utente girano con SCC `restricted-v2` per default, che **non permette** l'uso di `hostPath`. Per DaemonSet di sistema che necessitano di `hostPath`:

```
# Verificare la SCC assegnata a un ServiceAccount
oc get pod fluentd-xyz -o jsonpath='{.metadata.annotations.openshift\.io/scc}'

# Output atteso:
# privileged
```

```
# Assegnare SCC hostmount-anyuid a un ServiceAccount (solo se necessario)
oc adm policy add-scc-to-user hostmount-anyuid \
  -z my-logging-serviceaccount \
  -n openshift-logging
```

⚠ Attenzione: Su ARO, evitare assolutamente `hostPath` con `type: File` o `type: Directory` che puntino a path sensibili del nodo come `/etc`, `/var/lib/kubelet`, `/root`. Microsoft/Red Hat possono non supportare cluster con tali configurazioni.

emptyDir con medium: Memory su ARO

L'uso di `medium: Memory` consuma la memoria del nodo worker. Su ARO, i nodi hanno dimensioni fisse determinate dalla VM size scelta. Impostare sempre `sizeLimit`:

```
emptyDir:
  medium: Memory
  sizeLimit: 512Mi    # ← evita che un Pod esaurisca la RAM del nodo
```

```
# Verificare la memoria disponibile sui nodi ARO
az vm list --resource-group <aro-node-rg> \
  --query "[].{Name:name, Size:hardwareProfile.vmSize}" \
  --output table
```

Riepilogo dei Punti Chiave

- I volumi Kubernetes hanno ciclo di vita legato al **Pod** (non al container): sopravvivono ai restart del container ma non al delete del Pod, a eccezione dei PersistentVolume
 - `emptyDir` è la soluzione per dati temporanei e condivisione dati tra container nello stesso Pod; con `medium: Memory` usa la RAM (tmpfs)
 - `hostPath` è sconsigliato per applicazioni utente: richiede SCC elevate e rompe la portabilità; su ARO usarlo solo per DaemonSet di sistema
 - `configMap` e `secret` montati come volume permettono di aggiornare la configurazione senza riavviare il Pod (aggiornamento automatico con leggero ritardo)
 - `projected` volume combina più sorgenti in un unico mount point, utile per aggregare certificati, configurazione e token di ServiceAccount
 - Su ARO, impostare sempre `sizeLimit` su `emptyDir` (soprattutto con `medium: Memory`) per evitare esaurimento risorse del nodo
-

Domande di Verifica

1. Un'applicazione usa due container nello stesso Pod: il primo genera report CSV, il secondo li invia via email. Quale tipo di volume usare per condividere i file CSV tra i due container?

- A) `persistentVolumeClaim` con `ReadWriteMany`
- B) `emptyDir` ✓
- C) `hostPath`
- D) `configMap`

2. Un DaemonSet di logging deve leggere i log da `/var/log/containers/` su ogni nodo worker. Quale tipo di volume è appropriato?

- A) `emptyDir`
- B) `configMap`
- C) `hostPath` ✓
- D) `projected`

3. Quale affermazione su un volume `configMap` montato in un Pod è corretta?

- A) Il Pod deve essere riavviato perché le modifiche al ConfigMap siano visibili nel filesystem
- B) Il ConfigMap viene montato come file read-only, aggiornato automaticamente dal kubelet ✓
- C) Le chiavi del ConfigMap vengono montate come variabili d'ambiente, non come file
- D) Il ConfigMap viene copiato una sola volta al momento della creazione del Pod

4. Su ARO, quale SCC è richiesta per usare `hostPath` in un Pod utente?

- A) `restricted-v2` (quella di default)
- B) `nonroot`
- C) `privileged` o `hostmount-anyuid` ✓
- D) Nessuna SCC speciale è necessaria su ARO

5. Un volume `emptyDir` con `medium: Memory` ha quale caratteristica specifica?

- A) Usa Azure Disk per lo storage temporaneo
- B) I dati persistono dopo il delete del Pod perché sono scritti in memoria non volatile
- C) Usa la RAM del nodo (tmpfs), più veloce del disco ma consuma memoria del nodo ✓
- D) Può essere condiviso tra Pod diversi sullo stesso nodo